Correct computations, wrong conclusions: where errors entered an agent-run machine-learning experimentation pipeline

Abstract

Coding agents can now build and operate the software of a machine-learning study: data generators, training launchers, evaluation harnesses, and the reports that summarise them. When such a pipeline produces a wrong conclusion, where does the error come from, and what finds it? We studied a ten-day programme in which coding agents built and ran most of the pipeline for fine-tuning small language models to write programs, under one human research lead. From the version history, run records, two separate reviews, and a final audit, we screened every candidate defect. We retained 32 incidents that changed, or would have changed, a conclusion, a design decision, a launch, or an artifact. Two independent coders classified them with a fixed codebook and agreed on all but one category. The evaluation computations were reproducible: re-executing every saved held-out program reproduced every recorded outcome. The incidents divided sharply by kind. Code defects and environment failures announced themselves within hours to at most two days, through breakage, a reader of individual records, or a separate review. Errors of representation, where one artifact was taken to stand for another, took longer to find. Errors of inference and reporting broke nothing: nearly all persisted into the written record until a deliberate final audit found them. No incident was caught by the test suite, and several operational warnings addressed to "a human" were logged within a second of the automated actions they should have stopped, or were followed by them the next morning. In later comparisons run on the same pipeline, exact matching distorted two conclusions that a three-level scoring protocol with two blind, calibrated judges corrected. We derive checks for each kind. We argue that an agent-run pipeline needs an audit pass whose only instruction is to doubt its conclusions, scoring at more than one level, and guards that block rather than advise.

Keywords: empirical software engineering; machine-learning pipelines; coding agents; case study; research software; evaluation validity

1. Introduction

Late on the ninth day of our programme, we asked a coding agent to write solutions by hand for twenty problems from the two families on which every fine-tuned model scored zero. It was given the problem statements but not the reference answers; it began opening stored reference files during the run and was stopped, and none of its programs copies a reference. The project's runtime executed its programs and the project's own comparator scored them. They passed on 2 of 20. The record written at the time estimated that about 18 of the 20 computations were right, and a later item-by-item reading judged all twenty to state the reference value and verdict. The failures came from reference wording the statements did not determine, such as unit words and explanatory sentences, together with detail the solver added itself. Exact comparison rejected both.

This was not the first sign. Fifteen hours earlier, a reading of the previous run's records had concluded that 38 of 64 wrongly scored scheduling answers contained the right number and only paraphrased the verdict (a later check found that some of those numbers were counts rather than durations). The response had been to rewrite the reference answers rather than to question the comparator. Nothing in the comparator was a bug. It did exactly what it was written to do, and the defect was in what its output was taken to mean. The test suite could not see it, and it was found, twice, by reading individual items.

This paper asks where the consequential errors in an agent-built and agent-operated research pipeline came from, and what found them. The question matters because such pipelines are becoming common. Sculley and colleagues showed that the code implementing a model is a small part of a machine-learning system, and that most maintenance cost lives in configuration, data dependencies, and glue (Sculley et al., 2015). Amershi and colleagues found that machine-learning workflows differ from conventional software workflows in their data management and in how entangled their components are (Amershi et al., 2019). Breck and colleagues turned such observations into a rubric of tests for data, models, and infrastructure (Breck et al., 2017). Those studies

concern production systems built by human teams. Research pipelines built by coding agents are newer: the agents write the glue quickly and test it thoroughly, and they also write the reports that interpret its outputs.

We ask three questions. RQ1: what kinds of defect produced the consequential incidents? RQ2: how, and how quickly, was each kind detected? RQ3: which checks would detect each kind, and how did the checks that existed perform? Our contribution is an incident register for a complete, documented research programme, screened from all candidate sources and coded by two independent coders. From it we derive a description of how detection depends on the kind of defect, and checks derived from, and tested against, the observed failures.

2. The pipeline studied

2.1 Research goal

The programme fine-tuned small open code models, from 0.5 to 1.7 billion parameters, to translate word problems into programs in SOP Lang, a small dataflow language, rather than to answer them directly. A SOP Lang program declares named computations called wires. A wire's body can refer to another wire's value with `$name`, and those references define the execution order. A runtime executes the program and returns the requested value:

```
@slots literal
{"packs": 3, "perPack": 4}

@answer jsEval
return $slots.packs * $slots.perPack;
```

Here the model must extract the quantities into `slots` and choose the operation in `answer`; the runtime performs the arithmetic and returns 12.

2.2 Components

Coding agents wrote and maintained the pipeline's six components, with review by the research lead.

Data generators. Problem families derived from seven books of reasoning exercises and from procedural generators. Each generator emitted a statement, a reference program, and an answer. A verifier re-executed every program and perturbed its inputs to check that the answer depended on them.

Export and training. A trainer view of the dataset and a fine-tuning script that recorded, per run, a manifest with the base model, data hash, recipe, and timestamps.

Orchestration. Shell tooling that launched runs detached from interactive sessions so that they continued overnight, watched them, pruned old checkpoints, and started the next run when one finished.

Evaluation. A harness that served the selected checkpoint and generated a program for each held-out problem. It executed each program and classified the outcome as a parse failure, graph failure, execution error, answer mismatch, or match.

Diagnostics. Auxiliary suites: capability probes, contrastive minimal pairs, oracle-assisted conditions, in-context demonstrations, and execution-feedback retries.

Reporting. Per-run reports, an analysis log recording decisions, status notes for the research lead, and article drafts.

About 25 training runs were completed between 18 and 27 September 2026, most of them overnight; no work was recorded on two of those days. The version history holds 240 commits in the period.

3. Method

3.1 Design

This is an exploratory single-case study in the sense of Runeson and Höst (Runeson & Höst, 2009): one pipeline in its real context, studied after the fact through multiple data sources. The authors directed the programme, so we report the protocol in enough detail for the register and its coding to be checked independently.

3.2 Data sources

We used six data sources:

- the version history, with commit messages, diffs, and timestamps;
- run manifests for training and evaluation;
- per-item evaluation records, each with the generated program, its outcome, and the reference answer;
- the analysis log, in which decisions were recorded as they were made, together with the status notes and operational logs;
- two separate reviews, performed on days 5 and 8 by agent sessions instructed to review rather than build (the second survives only in a commit message);
- a final audit performed while preparing manuscripts, which recomputed reported numbers from item records and resolved model identities from manifest contents.

3.3 Screening and inclusion

Candidates came from four sources: the findings of the two reviews; every commit whose message mentions a fix, correction, bug, defect, repair, revert, remediation, mistake, or stale state; every row of the final audit's correction ledger; and incidents described in operational rules, the analysis log, and status notes. An incident is a defect in the pipeline or in the interpretation of its outputs that changed, or if not caught would have changed, one of the following: a written conclusion, a design decision, a training or evaluation launch, an artifact that was deleted or corrupted, or a reported measurement. We merged candidates describing the same defect. We excluded, with a recorded reason, failures whose only effect was delay, defects caught by a test or verification gate before use, cosmetic slips, and governance decisions.

Table 1 gives the screening funnel.

Table 1. Screening of candidate incidents.

Source	Candidates	Mapped to an incident	Excluded
Findings of the two separate reviews	17	9	8
Commits matching correction terms (of 240)	25	12	13
Rows of the final audit's correction ledger	31	18	13
Operational rules, analysis log, status notes, logs	32	25	7
Total	105	64	41

The 64 mapped candidates merge into 32 incidents. The register records for each incident when it entered, when it was first detected, by what route, its consequence, pointers to the evidence, and its links to other incidents.

3.4 Coding

Before coding, we fixed a codebook with four categories.

- **Code defect (C).** The code does not do what its own specification, documentation, or evident intent says.
- **Representation error (R).** The code works as specified, but one artifact is taken as evidence of something it does not represent. We record the seam at which this happens: a name taken for an identity, a score taken for the capability it indicates, an intervention on an input taken as isolating one factor, a record or signal taken for a system state, or one evaluation population taken for another.
- **Inference or reporting error (I).** The evidence was represented correctly, but the stated conclusion goes beyond it or miscounts it.
- **Environment failure (E).** A resource or platform failure outside the project's code.

When an incident fits two categories, the codebook assigns the one whose correction would have prevented it.

Two coders classified all 32 incidents independently, working from neutral descriptions of each incident and its consequence, without access to the register's other fields, to each other's work, or to this manuscript. Both coders were separate agent sessions; we discuss this choice in Section 5.4. They agreed on the category of 31 of 32 incidents (Cohen's $\kappa = 0.96$), and on category and seam together for 30 of 32 ($\kappa = 0.92$). We resolved the two disagreements by the codebook's rule and recorded the reasons.

4. Results

4.1 The evaluation computations were reproducible

Before describing what went wrong, we record what did not. We re-executed every saved generated program from every held-out evaluation of a trained checkpoint in the programme through the current evaluation code: extraction, parsing, dependency-graph validation, execution, and answer comparison. That is 13,330 records from 26 evaluation runs. Every re-derived outcome class and every executed answer matched the recorded one. The evaluation was deterministic and its records were complete. This does not mean the code was free of defects: nine incidents below are code defects. It means that the recorded measurements are exactly what the pipeline computed, so every error below lies in what was computed, in how it was interpreted, or in how the pipeline was operated, not in how records were kept.

4.2 RQ1: kinds of defect

Table 2 gives the 32 incidents by category, with examples.

Table 2. Incidents by category. Seams are given for representation errors.

Category	Incidents	Examples
Code defect	9	Retry attempts after the first never executed their programs; the probe scorer accepted an answer if its first number matched; demonstration selection read the hidden reference solution; evaluation read the current export instead of the run's own; checkpoint pruning deleted run manifests
Representation error	11	Name → identity (2): a run labelled "17b" read as a 17B model; servers not bound to the checkpoint under test. Score → construct (4): exact phrasing read as correctness; strict-format probe failures read as lost capability. Intervention → distribution (1): "oracle-assisted" prompts outside the training distribution read as isolating a stage. Record → state (2): a "series done" line, also written by failed runs, read as completion. Population → population (2): a changed held-out set compared across versions; missing completions counted as model failures
Inference or reporting error	9	A comparison across model families reported as a "size floor"; a gain credited to new commands that almost no gained item used; an unrecorded agent re-scoring reported as a measurement; a failure attributed to training data that no run had contained
Environment failure	3	Scripts in /tmp wiped by a machine reset; the disk filled during a checkpoint save; working files removed from the tree

Incidents occurred in every component. The largest groups were evaluation (10) and reporting (8), followed by diagnostics (5), orchestration (4), environment (3), training (1), and data generation (1).

4.3 RQ2: detection depends on the kind of defect

Table 3 cross-tabulates category against the route of first detection.

Table 3. Route of first detection by category (32 incidents).

First detected by	Code defect	Representation error	Inference or reporting	Environment
Reading individual records (working thread)	5	4	1	0
Operational failure	2	2	0	3
Separate review	2	2	0	0
Final audit	0	3	8	0
Test suite	0	0	0	0

The pattern is sharp. No code defect or environment failure survived to the final audit; they were found by breakage, by someone reading individual records, or by a separate review. The median time from entry to detection, where both could be dated, was about seven and a half hours for code defects (at most two days) and about three hours for the two datable environment failures. Representation errors were spread across all routes, with a median of about a day. Inference and reporting errors were almost never found during the programme: eight of nine were found only by the final audit, with a median of about forty hours. Five of the nine entered the record on the last two days, when the programme changed its aim from an accuracy target to a comparative claim. The one found during the programme was a regression diagnosis: the drop was first called a real signal,

then attributed to a distribution shift, and the second diagnosis was corrected four minutes after it was written, by a pass that read the fifteen failing answers.

No incident was first detected by the test suite, and none of the 32 could have been. The defects that tests do catch, such as a program that fails to compile or a function that returns the wrong type, were either caught before use and excluded by our inclusion rule, or were loud failures whose only cost was delay.

4.4 The largest cases

The comparator. Exact comparison of reference answers transcribed from the books began to hide correct work when the first run that could compute one family's solution was evaluated on day 8. Its scheduling programs computed the right duration and verdict in 64 of 100 cases, by a parser we wrote afterwards, and exact comparison accepted none. A reading of the records of the next run flagged the phrasing problem about ten hours later. The response was to compress the reference answers into value-bearing forms. The solver test fifteen hours after that confirmed that the comparator failed correct work. The problem did not end there. After the references were compressed, the model kept printing the answer sentence of a related family, and exact matching stayed at zero. For the other stuck family the zeros had a computational cause, not a phrasing one. Most of the zeros the programme spent effort on were real failures.

The model identity. A pipeline script named a run with a "17b" label, the base model's name without its decimal point, before a selection step had chosen that base. The next day the programme's roster recorded the run correctly as the 1.7B model, and the day after an article draft labelled it "1.7B base". On day 9 a status note read the label as a 17B model and used the result as decisive evidence for a data change, and a training run on that basis started half an hour later. That evening the programme adopted a comparative goal, and an article rewrite replaced the correct "1.7B base" with "17B base". The final audit found the error by printing each model's identity from manifest contents. The identity had been recorded correctly twice, and that did not prevent the error.

The assisted diagnostic. A diagnostic asked the same 60 problems four ways: the statement alone, with correct extracted values supplied, with the correct operation plan supplied, and with both. Supplying values or the plan barely changed accuracy (8, 9, 8 and 10 of 60). The analysis concluded that the difficulty lay in adapting a remembered plan, and that conclusion shaped the design of the next training run, a contrastive-pairs curriculum. Nine hours later, a follow-up on a second suite and a different model found problems with the instrument:

- The assisted conditions scored worse than the statement alone: 3 of 48 with values supplied, against 23 of 48 without.
- In none of 24 problem pairs did an assisted condition produce the same program as the plain one. The appended information used wording that occurred nowhere in training.
- Rewriting the appended block in training notation recovered part of the gap (8 of 24 with values supplied, against 12 of 24 without), but the programs still differed.

The record then kept the first conclusion's weaker part (supplying values bought little) and withdrew its stronger part (the conditions isolate a stage).

Sclar and colleagues show that formatting changes that should be irrelevant change model performance substantially (Sclar et al., 2024). A diagnostic that adds information to a prompt intervenes on the prompt's distribution as well as its content.

4.5 Cascades, and guards addressed to humans

Incidents clustered in two cascades.

The night of day 6. A machine reset had wiped operational scripts kept in /tmp, and they were restored to the repository that afternoon. The trainer's memory guard, which read free memory from a view that excluded reclaimable cache, stopped a run at step 160. While base models were downloading, the forced checkpoint save filled the disk and was corrupted. A disk guard existed: it warned from 20:35 UTC and logged that it was

stopping the trainer at 21:15 UTC. Its thresholds were below the size of one checkpoint save, and it did not stop downloads. The failed evaluation chain nevertheless wrote the "series done" line. The next morning a watcher read that line as completion. It logged "chain closed WITHOUT metrics — a human must look", and in the same second logged "launching" the next run, which started beside the interrupted one. Minutes later it pruned the interrupted run's checkpoints. The base model for that next run had been chosen by a comparison in which 322 of 705 requests had no completion. The selection gate had itself logged "NOT training — a human decides", and the choice was made the next morning.

Days 9 and 10. The comparator problem and the identity error combined with an unrecorded re-scoring and a mistaken diagnosis of one family's zeros, just as the programme changed its aim. A new data version and two training runs followed within hours. A pre-declared stop rule had required the comparator to be re-tested before any further training. It was superseded by the change of aim, and the re-test was never run.

Both cascades contain guards that existed and did not act. The disk guard acted too late, with the wrong thresholds. The gate and the watcher wrote messages addressed to a human reader, while the automation around them proceeded as if no message had been written.

4.6 Measuring the measurement

The largest representation error in the register was a comparator that required reference wording. After the programme, we used the same pipeline for a set of evaluation-only comparisons: untuned models answering directly, writing plain JavaScript, or writing an operator plan, and fine-tuned models re-evaluated on reworded problems. These comparisons allowed us to measure how much the scoring rule, rather than the systems, decides a reported difference. Every record was scored at three levels:

- **Exact:** the project's comparator.
- **Strict value:** a deterministic check that the last number stated equals the reference value, or that the numbers match as a multiset, and that any feasibility or yes/no verdict agrees.
- **Semantic:** exact, or accepted by both of two blind judges.

The judges were separate agent sessions that saw only the statement, the reference, and the candidate, with the producing model and condition hidden and the items shuffled. They judged 2,019 distinct candidate answers plus 20 injected answers known to be wrong. They agreed on 98.2% of the answers ($\kappa = 0.96$), accepted none of the 20 wrong answers, and accepted all 50 answers that a deterministic parser proves equivalent to the reference. Every verdict is retained with its reason. The calibration items sat in the first two passes; the third pass, which judged the larger models of Section 4.6, contained none, so its accuracy is inferred from the earlier passes and from the judges' agreement on it (616 of 619). Judging took three passes as items were added, fourteen agent sessions in all, and a few minutes of wall-clock time per pass. The judges were not infallible: both accepted answers to elapsed-time problems that omitted the unit the reference states, a leniency visible in the retained verdicts. Both judges were sessions of the same model family, so their agreement shows consistency, not independence of perspective.

Table 4 shows how the level changes what a reader would conclude.

Table 4. The same records scored at three levels. Counts of correct answers; the two composition rows use the same 55 problems.

Condition	Items	Exact	Strict value	Semantic
Fine-tuned 1.7B models, reworded scheduling problems (three runs)	45	0	45	45
Untuned 1.7B, plain JavaScript, held-out compositions	55	9	39	40
Untuned 1.7B, operator plan, held-out compositions	55	44	49	49
Fine-tuned 1.7B (final data), book problems	55	7	22	27
Untuned 1.7B, plain JavaScript, withheld stock problems	40	0	33	24
Fine-tuned 1.7B (compact targets), withheld stock problems	40	0	5	2

Three consequences follow for anyone instrumenting such a pipeline.

- **Exact matching can change a conclusion, not merely lower a score.** By exact matching, the operator plan beat plain JavaScript by 44 to 9, and the untuned and fine-tuned models tied at zero on the withheld stock problems. At the semantic level the first gap narrows from 35 answers to 9 (49 to 40), and the second becomes the largest difference in the comparison (24 to 2).
- **Deterministic value checks help, but they over-accept free-form verdicts.** On the withheld problems, the strict check accepted 33 answers that stated the right number of days. The judges reduced these to 24, because several answers attached the wrong ordering decision ("must be placed today: true") to the right number.
- **A judging protocol with retained verdicts, two independent judges, and calibration items is cheap.** The unrecorded re-scoring of the incident register (a meaning-based accuracy reported without verdicts) was a failure of protocol, not of the idea. The same idea, executed with a protocol, took minutes and produced evidence another reader can inspect.

The evaluation-only comparisons produced their own incidents, and they illustrate the paper's argument at small scale.

- **Different items per wording.** The first sample drew different items for each wording, because the seed hashed identifiers that included a wording suffix. The comparisons across wordings were therefore unpaired. A separate fact-check found this by counting shared identifiers, and the comparisons were re-scored on identical items.
- **Two wordings in one agent batch.** Two larger models were evaluated through agent sessions, each answering a batch of prompts. The first design put two wordings of the same problem in one batch, so an agent could carry an answer from one wording to the other. The answers were identical across wordings, but that was not proof of carry-over: after the re-run with separate sessions, the JavaScript answers of each model were still identical across wordings in 42 to 45 of 45 problems. The defect was found by reading the batch design, not the numbers. The re-run did not remove every pairing: four accepted batches still combine two answer forms of the same problems.
- **A scripted batch.** A review of the sessions' tool use found one session that had assembled its replies with a script rather than answering item by item. The batch was discarded, but the session transcripts were not retained, so the review itself cannot now be re-checked. This is the paper's point in miniature: a check whose evidence is not kept is an assertion.

All three defects produced plausible numbers, and none was caught by a test. They were found by checking what an artifact represented: whether the items were the same, and whether the answers were independent. The affected batches were discarded and re-run so that no session saw two wordings of a problem.

4.7 RQ3: checks per kind, and how the existing ones performed

Table 5 derives a check for each kind of defect and records how the corresponding existing check, if any, performed in the case.

Table 5. Checks by kind of defect, with the observed behaviour of the checks that existed.

Kind	Check	What the case shows
Code defect	A separate review with an instruction to find defects; reading a sample of records from every new run	All nine were found within hours to two days: five by reading records, two by breakage, two by the first review
Representation: name → identity	Generate every model reference from the manifest; fail any report whose stated identity disagrees	Printing the identity once did not prevent a later rewrite from overwriting it; only a blocking check would have
Representation: score → construct	Before relying on a comparator, score a competent solver's answers with it; score every comparison at more than one level; for judgement-based scores use two blind judges, calibration items, and retained verdicts	The solver test found the comparator problem, fifteen hours after records had flagged it; in the later comparisons, exact scoring distorted two conclusions that the retained-verdict protocol corrected (Section 4.6)
Representation: intervention → distribution	Show that a content-neutral version of a prompt intervention leaves outputs unchanged before interpreting it	The assisted conditions changed every program, which was measurable at once
Representation: record → state	Use result artifacts, not log lines, as completion signals; bind every run to hashed inputs	Adopted after the cascade; the "series done" line had already caused a deletion and a second trainer
Representation: population → population	Diff item identifiers, references, and programs before comparing evaluations; count missing responses separately	Both incidents were found only by the final audit
Inference or reporting	A separate audit pass instructed to find reasons each headline is false, reading item records, before the claim enters any plan	Found eight of nine; the ninth was found by reading records
Environment	Preflight gates on free disk and concurrent jobs, with thresholds above the largest single write, stopping downloads too	The existing guard fired and failed; it was strengthened afterwards
Any guard	Make guards block the automated action, rather than log a message for a human	One human-addressed warning was logged in the same second as an automatic launch, another within a second of a pruning; a third was followed the next morning by the launch it had declined

5. Discussion

5.1 What broke, and what did not

The central observation is an asymmetry in detection. Code defects and environment failures break things: a program crashes, a run stops, a result looks wrong at item level, and an agent or a reviewer notices within hours or, at most, days. Inference and reporting errors break nothing. A comparison across model families written as a "size floor", a gain credited to commands its items did not use, or an unrecorded re-scoring presented as a measurement: each looks like a finished result, and each was carried forward in status notes and drafts until a

process whose only task was to doubt them read the records. The computations themselves were reproducible down to every record. The consequential errors lay above them.

This is consistent with the concern Sculley and colleagues raise about entanglement and undeclared consumers (Sculley et al., 2015), shifted from code to claims. In an agent-run pipeline, a conclusion is consumed by the next plan, the next data change, and the next draft, usually without being re-derived.

5.2 Commission the doubt separately

The two separate reviews and the final audit found what the working thread did not, and the difference was their instruction. The working thread was asked to make progress, and a result that looked like progress was accepted. Kapoor and Narayanan document how errors such as leakage recur across many machine-learning studies (Kapoor & Narayanan, 2023); within one programme, the analogue is that a claim can be repeated many times without being tested once. In an agent-run pipeline a questioning pass is cheap, and our case suggests it is where most of the value lies. It should run before a claim enters a plan, not after a paper is drafted.

5.3 Guards must block

Human-addressed warnings did not work in a pipeline that ran overnight with no human present. "A human decides" and "a human must look" were written into logs that nobody read before the next automated step had acted. A guard in such a pipeline must prevent the action it warns about, and a condition that needs human judgement must stop the automation until a dated decision is recorded. The same holds for governance rules. A stop rule that does not name who may lift it can be dissolved by a change of aim without anyone deciding to lift it.

5.4 Threats to validity

Construct validity. The inclusion rule depends on documented consequences, so incidents without a recorded consequence are missing, and the count is a lower bound. "Would have changed" is a counterfactual judged after the fact.

Internal validity. The authors directed the programme and conducted the analysis. We mitigated this with a fixed codebook, a screening funnel covering every candidate source, and two independent coders. However, both coders were agent sessions rather than people, and they may share interpretive biases with each other and with the agents that operated the pipeline. Their high agreement shows that the codebook can be applied consistently. It does not show that its categories are the right ones.

Temporal confounding. Detection routes are confounded with time: the final audit happened after the programme ended, and the incidents it found are concentrated in the programme's last days.

External validity. This is one pipeline, one research lead, and one family of coding agents over ten days. We make no claim about the frequency of such incidents elsewhere, and we cannot compare agent-built with human-built pipelines.

Reliability. The checks in Table 5 are derived from the incidents they would catch. Their value on other pipelines is a hypothesis, which the case supports only where an equivalent check existed and its behaviour was observed.

6. Conclusion

In a research pipeline built and operated largely by coding agents, the recorded measurements were exactly reproducible, and many of the conclusions drawn from them were not sound. Code defects and environment failures announced themselves within hours to days. Errors in what an artifact was taken to mean were found more slowly. Errors of inference and reporting broke nothing, and almost all of them survived until an audit whose only job was to doubt. Tests caught none of the consequential incidents, and warnings written for a

human who was not there were logged beside the actions they should have stopped. Such a pipeline needs checks at the points where one artifact stands for another, scoring that does not let a comparator decide the conclusion, guards that block, and a separate, early, questioning pass over every claim that a plan or a paper will rely on.

Data availability and declarations

The replication package contains the version history, run manifests, item records, analysis log, reviews, and audit ledger. It also contains the incident register with its screening funnel, the codebook, both independent codings and their agreement, and the script that re-executes every saved program. Its location will be provided on submission. Coding agents were used throughout the programme studied, as the second coder pair, and to assist in drafting this manuscript. The authors checked every reported fact against the records. Author details, funding, and competing interests are to be completed before submission.

References

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software Engineering for Machine Learning: A Case Study. *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291-300. https://doi.org/10.1109/ICSE-SEIP.2019.00042

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. *2017 IEEE International Conference on Big Data (Big Data)*, 1123-1132. https://doi.org/10.1109/BigData.2017.8258038

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804. https://doi.org/10.1016/j.patter.2023.100804

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131-164. https://doi.org/10.1007/s10664-008-9102-8

Sclar, M., Choi, Y., Tsvetkov, Y., & Suhr, A. (2024). Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. *International Conference on Learning Representations*. https://openreview.net/forum?id=RIu5lyNXjT

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28. https://proceedings.neurips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html